

Riassunto OOP

Mara Grossi

July 2025

Contents

1	OO Programs	4
1.1	Principi SOLID	4
1.2	Compilazione ed esecuzione in Java	5
2	Java style rules	5
2.1	Variabili locali	5
2.2	Mutability	6
2.3	Identity ed equality	6
2.3.1	Shallow e deep equality	6
3	Classi	7
3.1	Visibilità	7
4	Types	7
4.1	Type checking	8
4.2	Overloading	8
5	Type Hierarchy	9
5.1	Inheritance	9
5.2	Principio di sostituzione di Liskov	10
5.3	Condizioni necessarie per Liskov	11
5.3.1	Signature rule	11
5.3.2	Methods rule	11
5.3.3	Properties rule	11
5.4	Polimorfismo	11
5.5	Conclusioni	12
6	Generic types	12
6.1	Generic methods	12
6.2	Type erasure	12
6.3	Bounded generic types	13
6.4	Wildcards	13
7	Assertions	13
7.1	Invarianti	13
7.2	Come gestire le precondizioni	14
7.3	Assertive and Defensive programming	14
7.4	Conclusioni	15

8	Exceptions	15
8.1	Partial Abstraction	15
8.1.1	Strategia 1: estendere la funzione	15
8.1.2	Strategia 2: exception	16
8.2	Exception	17
8.3	Checked and unchecked excpetions	17
8.4	Definire nuove exceptions	18
8.5	Try-catch	18
8.6	Quando e come usare le exceptions	19
8.7	Gestire le exception	19
8.7.1	Masking	19
8.7.2	Reflecting	19
9	Abstraction	19
10	Procedural abstraction	20
10.1	Contratto	20
10.2	Benefici	21
10.3	Proprietà	21
10.4	Partial abstraction	21
11	Data abstraction	22
11.1	Implementare un ADT	22
11.2	Problemi	22
11.2.1	Equality	22
11.2.2	Cloning	23
11.3	Specification di un ADT	23
11.4	L'abstraction invariant	23
11.5	L'abstraction function	23
11.6	Representation invariant	23
11.7	Effetti benefici	24
11.8	Esporre la representation	24
11.9	Preservare l'invariante	25
11.10	Operazioni	25
12	Iteration abstraction	25
12.1	Iterators	26
12.2	Definire iteratori	26
12.2.1	Implementazione	28
12.3	Dipendenze dell'iteratore sulla classe base	28
12.4	Problemi con gli iterators	29
13	Interfacce e abstract classes	29
13.1	Interfacce	29
13.2	Dependency inversion	30
13.3	Abstract classes	31
13.4	Struttura tipica	32
13.5	Abstract classes o interfacce?	32
14	Anonymous classes	32
15	Object oriented design	32
15.1	Design principle	33
15.2	OO Design	33
15.3	CRC technique	33

16 Design patterns	33
16.1 Factory	34
16.2 Abstract Factory	35
16.3 Builder	38
16.4 Observer	40
16.5 Decorator	42
16.6 Composite	44
16.7 Bridge	46

1 OO Programs

L'OO programming è un paradigma di programmazione basato sull'idea di organizzare il codice in oggetti. Un oggetto è un'unità che racchiude:

- Dati (detti **attributi** o proprietà) per descrivere lo stato dell'oggetto
- Funzioni (dette **metodi**) per definire il comportamento dell'oggetto

Concetti fondamentali:

1. **Classe**: è un “modello” o “stampino” per creare oggetti.
2. **Oggetto**: istanza concreta di una classe, con valori specifici.
3. **Incapsulamento**: nascondere i dettagli interni e fornire solo un'interfaccia pubblica.
4. **Ereditarietà**: una classe può “ereditare” proprietà e metodi da un'altra, evitando di riscrivere codice.
5. **Polimorfismo**: oggetti diversi possono rispondere in modi diversi allo stesso messaggio/metodo.

Esempio della pizza

- Classe: è il modello generale: dice quali ingredienti servono e come si prepara, ma non è una pizza vera, solo un'idea.
- Oggetto: è una pizza concreta, fatta seguendo la ricetta.
- Incapsulamento: tu ordini la pizza, ma non sai (o non puoi cambiare) come lui prepara l'impasto segreto.
- Ereditarietà: puoi avere una classe `PizzaVegetariana` che eredita da `Pizza` e aggiunge le verdure senza dover riscrivere tutto.
- Polimorfismo: se dici “fammi cuocere la pizza” a una Margherita o a una Diavola, entrambe “capiscono” l'istruzione, ma ognuna lo fa a modo suo.

1.1 Principi SOLID

I principi **SOLID** sono cinque linee guida per scrivere codice orientato agli oggetti che sia più pulito, flessibile e facile da mantenere.

S - Single Responsibility Principle (SRP)

Una classe deve avere una sola responsabilità. Ogni classe dovrebbe fare una cosa sola e farla bene.

- Classe `Fattura` che calcola l'IVA e stampa il PDF e invia l'email → NO
- Una classe `CalcolareIVA`, una `GeneratorePDF` e una `EmailSender` → SI

O - Open/Closed Principle (OCP)

Il codice deve essere aperto all'estensione, ma chiuso alla modifica. Puoi aggiungere nuove funzionalità senza toccare il codice esistente, evitando di rompere ciò che già funziona. Uso tipico: ereditarietà o interfacce per aggiungere nuovi comportamenti.

L – Liskov Substitution Principle (LSP)

Le sottoclassi devono poter sostituire le loro superclassi senza problemi. Se una funzione usa un oggetto della classe base, dovrebbe funzionare anche con una sua sottoclasse, senza comportamenti strani. Una sottoclasse `AnatraDiPlastica` che eredita da `Anatra` ma non può volare né fare “qua qua” rompe l'aspettativa → NO

I – Interface Segregation Principle (ISP)

Meglio tante interfacce specifiche che una gigante. Un client non deve essere costretto a implementare metodi che non usa.

- Interfacce piccole e mirate: `Stampabile`, `Salvabile`, `Condivisibile` → SI
- Un'unica interfaccia `Documento` con 20 metodi che non servono a tutti → NO

D – Dependency Inversion Principle (DIP)

Dipendi da astrazioni, non da implementazioni concrete. Il codice deve lavorare con interfacce o classi astratte, non con classi specifiche, così da poter cambiare facilmente l'implementazione.

- Un `PagamentoService` che usa un'interfaccia `MetodoPagamento` → SI
- Un `PagamentoService` che chiama direttamente `PayPalPagamento` → NO

1.2 Compilazione ed esecuzione in Java

In Java compilazione ed esecuzione sono due fasi distinte, la prima svolta dal compilatore Java (**javac**) e la seconda dalla Java Virtual Machine (**JVM**). Tu scrivi il codice sorgente in un file `.java`. `Javac` non crea codice macchina diretto per il sistema operativo, invece traduce il sorgente in bytecode, un linguaggio intermedio. Il bytecode viene salvato in file `.class`. La JVM:

- Carica le classi
- Verifica il bytecode
- Esegue il bytecode
- Gestisce la memoria tramite il Garbage Collector.

2 Java style rules

- Il nome dei package deve essere in camelCase con la prima lettera minuscola
- Il nome delle classi deve essere in CamelCase con la prima lettera maiuscola
- Il nome dei metodi e delle variabili devono essere in camelCase con la prima lettera maiuscola e i metodi devono iniziare con un verbo
- Racchiudere sempre tutti gli scope con `{}`
- Usare nomi significativi, evitare disinformazione
- Usare nomi pronunciabili e facilmente ricercabili nel codice
- Evitare di mettere nei nomi il tipo della variabile

2.1 Variabili locali

Come vengono gestite le variabili locali in memoria:

- **Stack**: contiene record di attivazione delle funzioni:
 - valori delle variabili locali
 - indirizzo di ritorno
 - valore di ritorno

Segue una politica LIFO

- **Heap:** memorizza variabili dinamiche, cioè allocate a runtime con `new`. Segue una politica garbage collection. La garbage collection è un meccanismo che gestisce automaticamente la memoria, liberando quella che non serve più.

Esempio:

```
1 int a[] = new int[5];
```

- Creo un array di 5 interi.
- `a` è una variabile locale che memorizza l'indirizzo dell'array.
- Nello stack `a` non contiene direttamente i 5 interi, ma l'indirizzo in cui si trova l'array nella heap.
- Nella heap viene effettivamente creato l'array di 5 elementi `a[0]`, ..., `a[4]`, inizializzati a 0.
- Se assegni `null` ad `a`, perdi il riferimento all'array nella heap.
- Ciò rende quell'area idonea per la garbage collection.

2.2 Mutability

È la capacità di un oggetto di cambiare stato (cioè modificare i suoi dati interni) dopo che è stato creato. Un oggetto è mutabile se il suo contenuto può essere modificato senza creare un nuovo oggetto. Esempio: `ArrayList`, `HashMap`, `StringBuilder`. Un oggetto è immutabile se il loro stato non cambia, ogni “modifica” in realtà crea un nuovo oggetto. Esempio: `String`, `Integer`, `LocalDate`. Ci sono anche gli shared objects: oggetti il cui indirizzo è salvato in più variabili.

2.3 Identity ed equality

`==` confronta i riferimenti in memoria (identity check), cioè controlla se due variabili puntano allo stesso oggetto.

```
1 String a = new String("ciao");
2 String b = new String("ciao");
3 System.out.println(a == b); // false
```

`equals()` confronta il contenuto logico degli oggetti.

```
1 String a = new String("ciao");
2 String b = new String("ciao");
3 System.out.println(a.equals(b)); // true
```

2.3.1 Shallow e deep equality

Shallow equality e deep equality indicano due modi diversi di confrontare oggetti.

- **Shallow equality:** confronta solo i riferimenti degli oggetti (cioè se puntano alla stessa area di memoria). L'operatore `==` fa shallow equality. Due variabili sono “uguali” solo se sono lo stesso oggetto, non importa se hanno lo stesso contenuto.
- **Deep equality:** confronta il contenuto degli oggetti, non solo i riferimenti. Si ottiene ridefinendo il metodo `equals()`. Se l'oggetto contiene altri oggetti o array, deep equality implica confrontare ricorsivamente anche quelli.

3 Classi

Una classe è una categoria che definisce:

- variabili di istanza
- **costruttori**
- altri metodi, quali **accessors** e **modifiers**

Gli accessors sono metodi pubblici che permettono di accedere ai valori delle variabili (attributi) private di una classe senza modificarne lo stato. Sono anche detti **Getter**. I modifiers sono metodi che modificano il valore di un attributo privato. Possono includere controlli o vincoli prima di assegnare un nuovo valore. Sono anche detti **Setter**.

Esempio

```
1  public class Person
2  {
3      private String name; // variabile di istanza
4
5      // Getter = accessor
6      public String getName()
7      {
8          return name;
9      }
10
11     // Setter = mutator
12     public void setName(String newName)
13     {
14         name = newName;
15     }
16 }
```

3.1 Visibilità

- **Public** → Accessibile ovunque
- **Private** → Accessibile solo all'interno della stessa classe
- **Protected** → Accessibile nella stessa classe, sottoclassi, e stesso package
- **Default** → Accessibile nel package

Dove possibile, usare **private**. Le variabili di istanza (o fields) vanno sempre dichiarate **private**. Solo le costanti possono essere **public**.

4 Types

Java fornisce dei dati di base, ovvero i tipi primitivi, che non sono oggetti, ma valori semplici e diretti, utilizzati per rappresentare numeri, caratteri o valori logici. Ogni tipo primitivo ha una classe wrapper corrispondente, ovvero una classe oggetto che “incapsula” il valore primitivo. Queste classi si trovano nel package `java.lang` e forniscono metodi utili per operazioni come parsing, confronto, conversione, ecc.

Primitive type	Dimension	Wrapper class
byte	8 bit	Byte
char	16 bit	Character
int	32 bit	Integer
short	16 bit	Short
long	64 bit	Long
float	32 bit	Float
double	64 bit	Double
boolean	1 bit	Boolean

Attraverso l'**autoboxing** i tipi primitivi vengono automaticamente convertiti nella loro wrapper class. L'autoboxing può risultare pericoloso perché può introdurre problemi di performance, bug logici e comportamenti inattesi. Java è fortemente tipato: ogni variabile ha un tipo, ogni metodo ha la sua signature, il compilatore controlla la correttezza dei tipi.

4.1 Type checking

Il compilatore deduce il tipo apparente di un'espressione al compile time. L'**apparent type** è il tipo dichiarato staticamente da una variabile, cioè il tipo che il compilatore Java "vede" durante la compilazione, indipendentemente dal tipo reale dell'oggetto a runtime. La JVM manipola il tipo reale al runtime. JVM e compilatore assicurano il type safety:

- possono verificarsi errori di non corrispondenza tra tipi
- non si verificano out-of-bounds errors

Esempio

```

1  class Animale
2  {
3      void verso()
4      {
5          System.out.println("Suono generico");
6      }
7  }
8
9  class Cane extends Animale
10 {
11     // @Override
12     void verso()
13     {
14         System.out.println("Bau Bau");
15     }
16 }
17
18 public class Main
19 {
20     public static void main(String[] args)
21     {
22         Animale a = new Cane(); // Apparent type = Animale, Real type =
           Cane
23         a.verso(); // Output: Bau Bau
24     }
25 }
```

4.2 Overloading

L'overloading di un metodo avviene quando in una stessa classe si definiscono più metodi con lo stesso nome, ma firma diversa, ovvero con:

- un numero diverso di parametri,
- tipi diversi di parametri,
- oppure ordine diverso dei parametri.

Non conta il tipo di ritorno del metodo per distinguere gli overload. Il compilatore sceglie il metodo giusto in base ai parametri passati al metodo. Il **dispatching** è il meccanismo con cui viene scelto quale metodo eseguire a runtime o a compile-time, in base al tipo dell'oggetto o dei parametri. Si basa sul tipo apparente degli argomenti e il tipo reale delle istanze.

5 Type Hierarchy

Vogliamo creare un sistema di classi che condividono alcune proprietà, come stato e/o comportamento. Per questo definiamo una type hierarchy. In cima c'è il general type. I subtypes lo perfezionano:

- fornendo implementazioni alternative
- fornendo metodi aggiuntivi

L'**inheritance** è il meccanismo che permette un uso flessibile di una gerarchia.

5.1 Inheritance

Permette a una classe (subclass) di ereditare campi e metodi da un'altra classe (detta superclass).

Esempio

```

1  class Animale
2  {
3      void mangia()
4      {
5          System.out.println("L'animale mangia.");
6      }
7  }
8
9  class Cane extends Animale
10 {
11     void abbaia()
12     {
13         System.out.println("Il cane abbaia.");
14     }
15 }
16
17 class Gatto extends Animale
18 {
19     void miagola()
20     {
21         System.out.println("Il gatto miagola.");
22     }
23 }
24
25 public class Main
26 {
27     public static void main(String[] args)
28     {
29         Cane c = new Cane();
30         Gatto g = new Gatto();
31         c.mangia(); // Metodo ereditato
32         g.mangia(); // Metodo ereditato
33         c.abbaia(); // Metodo della sottoclasse

```

```
34     }
35 }
```

5.2 Principio di sostituzione di Liskov

”Ogni oggetto di una classe derivata deve poter sostituire un oggetto della classe base senza alterare il corretto funzionamento del programma.” In altre parole: se **S** è una sottoclasse di **T**, allora oggetti di tipo **T** possono essere sostituiti da oggetti di tipo **S** senza rompere il comportamento atteso del programma. Il compilatore Java assicura che metodi del supertype siano ereditati dal subtype; perciò il principio di sostituzione resiste per la parte sintattica. D'altra parte il compilatore non può assicurare che il contratto dei metodi ridefinito dal subtype sia allineato con quello del supertype.

Esempio

```
1  class Rettangolo
2  {
3      protected int larghezza;
4      protected int altezza;
5
6      public void setLarghezza(int larghezza)
7      {
8          this.larghezza = larghezza;
9      }
10
11     public void setAltezza(int altezza)
12     {
13         this.altezza = altezza;
14     }
15
16     public int getArea()
17     {
18         return larghezza * altezza;
19     }
20 }
21
22 class Quadrato extends Rettangolo
23 {
24     // @Override
25     public void setLarghezza(int lato)
26     {
27         this.larghezza = lato;
28         this.altezza = lato;
29     }
30
31     // @Override
32     public void setAltezza(int lato)
33     {
34         this.larghezza = lato;
35         this.altezza = lato;
36     }
37 }
```

Abbiamo una violazione del principio di Liskov. Supponiamo di avere:

```
1  public void stampaArea(Rettangolo r)
2  {
3      r.setLarghezza(5);
4      r.setAltezza(10);
5      System.out.println(r.getArea());
6  }
```

Se `r` è un `Rettangolo`, stamperà 50 (corretto). Ma se `r` è un `Quadrato` (sottoclasse), l'impostazione di larghezza e altezza sarà legata, quindi non avrai area 50, ma 100, rompendo l'aspettativa del codice.

5.3 Condizioni necessarie per Liskov

Per essere un buon subtype di un supertype, una subclass deve soddisfare:

- **signature rule:** i metodi della subclass devono essere compatibili
- **methods rule:** le subclass deve richiedere meno e promettere di più
- **properties rule:** le subclass devono preservare tutte le proprietà della superclass

5.3.1 Signature rule

L'oggetto del subtype deve avere tutti i metodi del supertype. I metodi devono essere compatibili: uguale signature, con l'eccezione che il subtype potrebbe lanciare meno exception types. Il compilatore Java assicura questa regola.

Sulle pre e post condizioni

Una condizione P è un predicato $P(S)$ su tutti i possibili stati di esecuzione del programma; può essere una pre o post condition, un invariante, un'assunzione. Lo stato di esecuzione è l'insieme dei dati che descrivono completamente l'attuale situazione di un programma.

5.3.2 Methods rule

Un metodo di un subtype potrebbe:

- indebolire una precondition: richiede meno, è più generale.
- rafforzare una postcondition: promette di più

Java non può assicurarli.

Method precondition rule

Un subtype può indebolire una precondition ogni volta che la chiamata al metodo del supertype sia legale, allora anche la chiamata al subtype lo sarà.

Method postcondition rule

Un subtype può rafforzare una postcondition ogni volta che la chiamata al metodo del supertype sia legale, allora chiamare un metodo di una subclass otterrà almeno lo stesso effetto del supertype, e forse qualcosa in più.

5.3.3 Properties rule

La subclass non deve alterare le invarianti stabilite dalla superclass e le sue proprietà.

5.4 Polimorfismo

È l'abilità di qualcosa di avere più forme. In questo contesto, lo stesso metodo o interfaccia può avere implementazioni diverse, a seconda dell'oggetto su cui viene chiamato. In Java possono essere polimorfici:

- metodi: attraverso il dispatching (in base alla signature si decide quale metodo usare se questi hanno la stessa signature)
- oggetti: un'istanza di un supertype può essere castata in un'istanza di un subtype

5.5 Conclusioni

Una famiglia di un tipo mette in chiaro quali sono le relazioni tra tipi. L'abstraction lavora sull'intera famiglia. Possono essere aggiunti nuovi tipi: ci saranno nuove abstractions, non solo nuove implementazioni. Se il principio di Liskov non viene raggiunto, usare delegation + association invece di hierarchy + inheritance.

6 Generic types

Le classi generiche in Java sono classi che possono lavorare con tipi di dati diversi senza dover essere riscritte per ogni tipo. Sono definite usando parametri di tipo, che vengono specificati solo quando si crea un'istanza della classe. `List<String> list = new ArrayList<String>()` piuttosto che `List list = new ArrayList()`. Perché lo facciamo? Per sbarazzarci del type casting, per permettere type checking più forti da javac e per riusare codice.

```
1 List list s= new ArrayList();
2 list.add("hello");
3 String s = (String) list.get(0);
```

Il compilatore non può controllare se il type casting sia safe. Potremmo avere delle exceptions al runtime e sarebbe più difficile da testare.

```
1 List<String> list s= new ArrayList<String>();
2 list.add("hello");
3 String s = list.get(0);
```

Non c'è bisogno di casting, il type checking è completamente effettuato al compile time e non c'è bisogno di test.

```
1 List<String> list s= new ArrayList<>();
2 list.add("hello");
3 String s = list.get(0);
```

Type inference: è la capacità del compilatore di dedurre automaticamente i tipi generici in base al contesto, senza che tu li specifichi esplicitamente. Nota: `Integer` è un subtype di `Number`, ma `List<Integer>` non è un subtype di `List<Number>`.

6.1 Generic methods

Sono metodi che dichiarano uno o più parametri di tipo (type parameters) nella propria definizione, così da poter lavorare con tipi diversi mantenendo il controllo di tipo (type safety) senza dover scrivere lo stesso codice più volte.

Esempio

```
1 public <T> void stampa(T valore)
2 {
3     System.out.println(valore);
4 }
```

`<T>` è il parametro di tipo, `T valore` è un argomento di quel tipo.

6.2 Type erasure

Javac assicura che al runtime non ci siano generics:

- tutti i type parameters sono rimpiazzati da argomenti con un tipo compatibile
- obbediscono al vincolo delle wildcards `<?...>`

Non ci sono overhead al runtime.

6.3 Bounded generic types

I bounded generic types (tipi generici limitati o vincolati) sono una funzionalità dei generics che restringe i tipi ammessi per un parametro generico usando la parola chiave `extends` (o `super` nei wildcard). Permette al compilatore di fare type checking sui type arguments.

Esempio

```
1 public interface Pair<K extends Comparable, V>
2 {
3     public K getKey();
4     public boolean hasLargestKey();
5     public V getValue();
6 }
```

6.4 Wildcards

Le wildcards sono simboli speciali usati nelle generics per indicare un tipo generico sconosciuto o con vincoli flessibili. Wildcard: `<?..>`. Nota: `List<A>` è un subtype di `List<?>`.

`public static void printList(List<Object> list)` accetta argomenti il cui tipo è `List<Object>`. Quello che ci serve è `public static void printList(List<?> list)`.

7 Assertions

Le assertion in Java sono uno strumento di debugging che permette di verificare che una certa condizione sia vera in un punto specifico del programma. Se l'asserzione fallisce, il programma lancia un `AssertionError`, utile per individuare bug durante lo sviluppo. La valutazione della condizione normalmente è disabilitata dalla JVM, quindi le assertion vengono usate durante lo sviluppo e il testing, purché siano abilitate. Se abilitate, sono utili perché materializzano alcune assunzioni, se disabilitate non si ha una penalità in fatto di prestazioni. Quando **non usare** le assertions:

- non dovrebbero essere usate per controllare tutte le precondizioni dei metodi; in particolare quelle dei metodi pubblici.
- non dovrebbero essere usate con espressioni booleane che hanno effetti collaterali, ovvero quando l'espressione è un'invocazione di un metodo che ha effetti su qualche oggetto.

Quando **usare** le assertions:

- per controllare precondizioni di argomenti e oggetti di metodi privati perché il loro contesto d'uso è limitato e una volta che la classe è stata testata, le assertion non sono più utili
- per controllare invarianti di qualunque metodo perché è utile durante lo sviluppo e le invarianti sono assunzioni che il programmatore fa
- per controllare postcondizioni dei metodi: i constrain effect e return nel contratto, le postcondizioni sono assunzioni che il programmatore fa sul comportamento del suo codice.

In generale: una assertion violata è un bug.

7.1 Invarianti

Un invariante è una proprietà o condizione che rimane sempre vera durante l'esecuzione di un programma, o durante una parte specifica del suo ciclo di vita, indipendentemente dai cambiamenti di stato. Come gestirle con le assertion:

```
1 if (i % 3 == 0)
2 {
3     ...
4 } else if (i % 3 == 1)
```

```

5      {
6      ...
7      } else // We know (i % 3 == 2)
8      {
9      ...
10     }

```

Il commento esprime un'invariante, dovrebbe diventare un'assertion:

```

1      if (i % 3 == 0)
2      {
3      ...
4      } else if (i % 3 == 1)
5      {
6      ...
7      } else
8      {
9          assert (i % 3 == 2);
10         ...
11     }

```

Ora l'invariante è diventata eseguibile, il codice ha un built-in test. Altro esempio: al posto di

```

1      void foo()
2      {
3          for (...)
4          {
5              if (...)
6                  return;
7          }
8          // Execution should never reach this point!!!
9      }

```

facciamo:

```

1      void foo()
2      {
3          for (...)
4          {
5              if (...)
6                  return;
7          }
8          assert false: "Execution should never reach this point!";
9      }

```

7.2 Come gestire le precondizioni

Argomenti di metodi pubblici dovrebbero essere testati con le exceptions:

- perché un'exception comunica ai chiamanti cosa è successo
- se venissero usate le assertions, allora, se disabilitate, allora non farebbero più quello che dovrebbero
- in generale, le exceptions sono un metodo più generico e flessibile per supportare il contratto

7.3 Assertive and Defensive programming

Defensive Programming

Obiettivo: rendere il codice resistente agli errori, robusto, anche se l'utente o un'altra parte del sistema si comporta in modo scorretto. Come funziona:

- Si assume che qualcosa potrebbe andare storto
- Si usano controlli espliciti per validare l'input e gestire ogni possibile eccezione.

Strategie:

- Graceful degradation: il programma produce alcune approssimazioni del comportamento atteso
- Strict behaviour: il programma si ferma con un messaggio d'errore significativo

Assertive Programming

Obiettivo: verificare le ipotesi fatte dal programmatore sul comportamento interno del sistema, assumendo che l'uso sia corretto. Come funziona:

- Si asserisce che certe condizioni siano sempre vere durante lo sviluppo
- Usa assert per validare stati interni o ipotesi logiche
- Le assert possono essere disattivate a runtime (quindi non si usano per controlli critici)

7.4 Conclusioni

Usare le exceptions per controllare le precondizioni di metodi pubblici:

- definirle nei contratti
- usarne di già esistenti quando possibile
- usare le checked quando possibile

Usare le assertions per materializzare qualsiasi assunzione:

- incluso precondizioni di metodi privati, postcondizioni e invarianti
- possibilmente in aggiunta alle exceptions
- Importante: il codice di produzione deve essere lo stesso

8 Exceptions

8.1 Partial Abstraction

Non sono sicure, uno deve conoscere e ricordare il contratto e assicurarsi che valga nel contesto specifico dove l'abstraction è usata. Un'astrazione parziale lascia al client il compito di soddisfare il vincolo require. Sono appropriate solo quando il contesto d'uso è limitato o quando l'efficienza lo richiede. Come gestirle?

- trasformarle in funzioni totali creando nuovi outputs ad-hoc
- usando try/catch/throws per segnalare invocazioni eccezionali
- usando gli assert per segnalare assunzioni violate

8.1.1 Strategia 1: estendere la funzione

Quello che facciamo è trasformare funzioni parziali in totali. In generale è la strategia meno efficiente, ci sono pochi casi in cui ha senso usarlo.

Nascondiamo l'errore estendendo il range

Nascondiamo gli errori nei risultati estendendo il result set, quindi gli errori si possono propagare.

```
1  /**
2   * Compute the factorial of n
3   * @param n
4   * @return n! if n >= 0; -1 otherwise
5   */
6
7  public static long factorial (int n)
```

In questo caso, il chiamante non può usare

```
1  z = x + MyNumbers.factorial(y)
```

ma deve scrivere

```
1  long x1 = MyNumbers.factorial(y)
2  if (x1 > 0) z = x + x1; else ...
```

A volte questa strategia non è possibile perché è facile fare errori ed è poco sicuro.

Nascondiamo l'errore senza estendere il range

Nascondiamo l'errore nel risultato senza estendere il result set, quindi gli errori sono nascosti. Il chiamante può usare

```
1  z = x + MyNumbers.factorial(y)
```

Questo porta a funzioni non iniettive, gli errori possono portare errori da altre parti, quindi è poco sicuro.

8.1.2 Strategia 2: exception

È un meccanismo per affrontare situazioni eccezionali. Non influenza il result set, ma influenza l'execution flow:

- terminazione normale
- terminazione eccezionale

Il chiamante deve gestire le exceptions: le può rilevare e decidere come gestirle. Vantaggi:

- separazione tra "normal" code ed "error-handling" code
- rappresentazione esplicita degli errori: raggruppiamo quelli simili
- supporto alla propagazione degli errori

Esempio di specification per le exceptions:

```
1  /**
2   * Compute the factorial of n
3   * @param n
4   * @return n!
5   * @throws NonPositiveArgumentException if n < 0
6   */
7
8  public static long factorial (int n) throws NonPositiveArgumentException
```

Non c'è più il vincolo require, `factorial()` ora è una funzione totale con due tipi di risultati:

- normale: `long`
- eccezionale: `NonPositiveArgumentException`

Un metodo può lanciare anche più exception insieme.

8.2 Exception

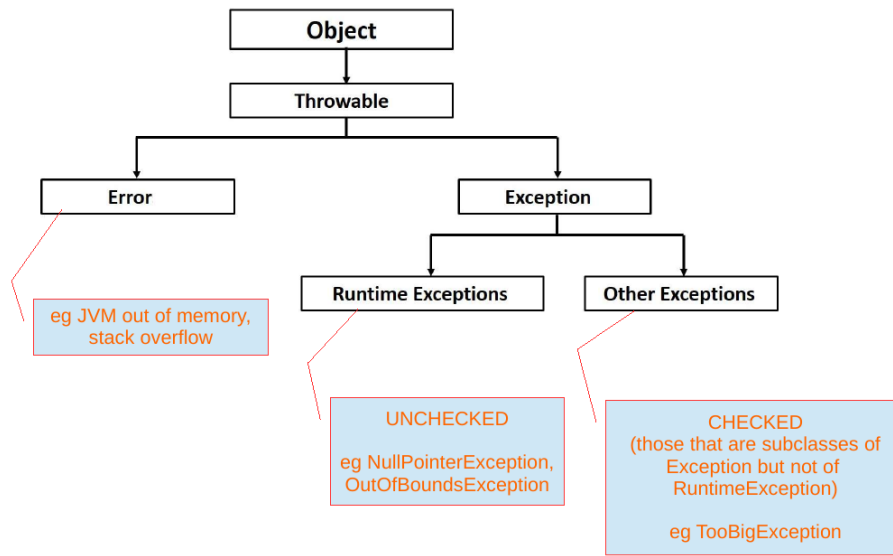


Figure 1: Type hierarchy di `Throwable`

Le exception possono essere predefinite o possono essere definite all'occorrenza. Sono normali classi/tipi e devono estendere `Exception` o `RuntimeException`. Le eccezioni sono oggetti che contengono informazioni sull'errore o sulla condizione anomala che si è verificata. Questi oggetti derivano dalla classe base `Throwable` e dispongono di metodi utili per ottenere dettagli e gestire l'errore:

- `public String getMessage():` restituisce una stringa con il messaggio associato all'eccezione (spesso impostato nel costruttore).
- `public Throwable getCause():` restituisce la causa (un'altra eccezione) che ha generato questa eccezione, se presente.
- `public void printStackTrace():` stampa sullo standard error la traccia dello stack, utile per il debug.
- `public StackTraceElement[] getStackTrace():` restituisce un array di `StackTraceElement` per analizzare programmaticamente la traccia.

8.3 Checked and unchecked exceptions

Una checked exception è un'eccezione che il compilatore obbliga a gestire esplicitamente, o tramite un blocco `try-catch` o dichiarandola con `throws` nella firma del metodo. Caratteristiche:

- Derivano da tutte le sottoclassi di `Exception`, eccetto quelle che derivano da `RuntimeException`.
- Controllo a compile-time: se un metodo può generare una checked exception e non la gestisci, il codice non compila.
- Obbligo di gestione: serve a forzare il programmatore a pensare a come gestire la condizione anomala.
- Tipico uso: errori recuperabili o previsti.

Le unchecked exceptions in Java sono eccezioni che il compilatore non obbliga a gestire. Caratteristiche:

- Derivano da `RuntimeException` e dalle sue sottoclassi.
- Controllo a runtime: il compilatore non verifica la loro gestione; se si verificano, causano l'interruzione del programma a meno che tu non le catturi.

- Tipico uso: errori di programmazione o condizioni che normalmente non dovrebbero accadere se il codice è corretto.
- Non obbligatorie da gestire: puoi intercettarle, ma spesso si preferisce lasciarle “risalire” lo stack, perché indicano bug o logica errata.

8.4 Definire nuove exceptions

Il nome della nuova exception deve terminare in “Exception”. Il messaggio e il dato devono permettere alla persona che legge l’exception di capire cosa sia successo, quando, perché e cosa dev’essere fatto.

Esempio

```

1 // Eccezione personalizzata (checked)
2
3 public class DatiNonValidiException extends Exception
4 {
5     public DatiNonValidiException(String messaggio)
6     {
7         super(messaggio); // Passa il messaggio alla superclasse Exception
8     }
9 }
10
11
12 // Eccezione personalizzata (unchecked)
13
14 public class ValoreNegativoException extends RuntimeException
15 {
16     public ValoreNegativoException(String messaggio)
17     {
18         super(messaggio);
19     }
20 }
```

8.5 Try-catch

Cosa fare quando occorre un’exception? Le exception possono portare a trasferimenti eccezionali del controllo che possono essere difficili da capire. Molti bug sono causati dal modo in cui le exceptions vengono gestite. Per questo usiamo il blocco **try-catch** che serve a gestire le eccezioni in modo che il programma non si interrompa bruscamente quando si verifica un errore. Come funziona:

- **try** racchiude il blocco di codice “a rischio” che potrebbe generare un’eccezione.
- **catch** intercetta l’eccezione lanciata e contiene il codice per gestirla.
- **finally** è opzionale e viene sempre eseguito dopo **try** o **catch**

Esempio (in base all’esempio precedente):

```

1 public class TestChecked {
2     public static void main(String[] args)
3     {
4         try
5         {
6             validaDati("");
7         } catch (DatiNonValidiException e)
8         {
9             System.out.println("Errore: " + e.getMessage());
10        }
11    }
12 }
```

```

13     static void validaDati(String nome) throws DatiNonValidiException
14     {
15         if (nome == null || nome.isEmpty())
16         {
17             throw new DatiNonValidiException("Il nome non puo' essere
18                 vuoto");
19         }
20     }

```

8.6 Quando e come usare le exceptions

Si usano per:

- errori imprevisti durante l'esecuzione
- condizioni anomale ma gestibili
- separare il flusso normale da quello di errore
- delegare la gestione
- quando il problema non è recuperabile senza intervento

Quando possibile, riutilizzare exceptions già esistenti perché sono più semplici da capire per gli altri e sono già state testate. Se si pensa che un client possa riprendersi da una exception, meglio farla checked; altrimenti farla unchecked. Tutte le exceptions devono essere menzionate nel contratto:

- il chiamante deve conoscere sotto quali circostanze l'exception può occorrere
- il chiamante deve conoscere anche cosa succede in questi casi agli oggetti mutabili che sono normalmente modificati

Rimpiazzano il vincolo require nel contratto.

8.7 Gestire le exception

8.7.1 Masking

Il chiamante gestisce l'exception direttamente: sistema il problema, notifica qualcuno e continua.

8.7.2 Reflecting

- Il chiamante gestisce le exception facendo qualche lavoro ma non tutto il lavoro. Fa qualcosa e poi lancia la stessa o un'altra exception al chiamante.
- Il chiamante non fa nulla, se un'exception occorre quando si chiama un metodo, il chiamante la lancia implicitamente al suo chiamante. Il contratto del chiamante deve menzionare l'exception.

9 Abstraction

Serve per nascondere i dettagli complessi dell'implementazione e mostrare solo le funzionalità essenziali di un oggetto o di una classe. Abbiamo vari modi per ottenere le abstraction:

1. **Abstraction by specification**
2. **Abstraction by parametrization**
3. **Procedural abstraction**
4. **Data abstraction**
5. **Iteration abstraction**

10 Procedural abstraction

Una procedura è una mappa dagli input agli output con possibili modifiche all'input. Possiamo ottenerla tramite parametrization, riutilizzando funzioni con dati diversi, e attraverso la specification, specificando per un metodo il contratto e separando **cosa** un metodo fa vs **come** lo fa.

10.1 Contratto

Definiamo un **contratto**, ovvero un insieme di regole che definiscono il comportamento atteso di una classe, metodo o interfaccia. Serve per garantire che chi implementa o usa quel codice sappia cosa aspettarsi e cosa rispettare. Precondizioni:

- Proprietà che devono essere vere quando il metodo viene chiamato affinché questo sia ben definito.
- Si riferisce agli argomenti passati al metodo
- e/o allo stato dell'istanza

Postcondizioni:

- Proprietà che devono essere vere alla fine dell'esecuzione del metodo (con l'assunzione che le precondizioni siano soddisfatte)
- Si riferiscono all'output del metodo
- e/o agli oggetti passati come argomenti che sono stati modificati
- e/o agli stati modificati delle istanze

Esempio

```
1  /**
2   * @param a: the array to partition; REQUIRE a.length > 0; MODIFY a
3   * @param i: starting index in a; REQUIRE 0 <= i < j < a.length.
4   * @param j: ending index in a;
5   * @return res, an index in array a such that each element in a_post[i,
6   *   ..., res]
7   * is smaller or equal to each element in a_post[res+1, ..., j]. MODIFY a
8   */
9  private static int partition (int[] a, int i, int j)
10 {
11     int current = a[i];
12
13     while(true)
14     {
15         while (a[j] > current)
16         {
17             j--;
18         }
19
20         while (a[i] < current)
21         {
22             i++;
23         }
24
25         if (i > j)
26         {
27             return j;
28         } else
29         {
```

```

30         swap(a, i, j);
31         j--;
32         i++;
33     }
34 }
35

```

Per classi/interfacce deve essere specificata la mission: cosa "sa" e cosa è capace di fare.

Esempio

```

1  public class Arrays
2  {
3      /**
4       * The mission of this class is to provide a number of
5       * standalone procedures that can be useful for manipulating arrays
6       * of ints.
7       */
8  }

```

La specification deve essere fatta prima dell'implementazione.

10.2 Benefici

- Locality: facilitare la comprensione, navigazione e manutenzione del codice, riducendo la necessità di saltare tra file, classi o moduli lontani.
- Modifiability: è la facilità con cui il software può essere modificato per correggere errori, adattarsi a nuovi requisiti o migliorare.

10.3 Proprietà

- Minimamente vincolata: il comportamento dell'abstraction dovrebbe essere vincolata solo se strettamente necessario o per lasciare maggiore libertà all'implementatore
- Non vincolata: per alcuni input il comportamento potrebbe risultare indefinito: dovrebbe essere specificato nel contratto
- Generalità: un'abstraction è più generale se può gestire una maggiore classe di inputs
- Deterministica: se chiamata più volte sullo stesso input, deve produrre lo stesso risultato
- Semplicità

10.4 Partial abstraction

Non è sicura, uno deve conoscere e ricordare il contratto. Lascia al client il compito di soddisfare i requisiti di un metodo. È appropriato usarle in contesti limitati o per motivi di efficienza. Modi per risolvere il problema:

- trasformare le funzioni parziali in totali creando degli outup ad-hoc
- usando try/catch/throw per segnalare exceptions
- usando gli assert per segnalare violazioni

11 Data abstraction

L'obiettivo è ottenere una struttura del programma che sia in corrispondenza con la struttura del problema. Per esempio, supponiamo di voler creare una classe per una macchina del caffè, questa:

- consiste in componenti come: contenitore dell'acqua, filtri, pompe, valvole, ecc
- permette operazioni come: `heatWater()`, `makeEspresso()`, `setupCoffeeFilter()`, ecc

La macchina del caffè è un nuovo data type. Un ADT è un modello concettuale che definisce dati e operazioni su quei dati, senza specificare l'implementazione. In altre parole, un ADT descrive cosa fa una struttura dati, non come lo fa.

Esempio

```
1  public class IntSet
2  {
3      /** This class provides an ADT for sets of int.
4       * IntSet is mutable, unbounded. */
5
6      /** @return: a new set, empty. */
7      public IntSet(){...}
8
9      /** @param: elts. Can be null.
10     * @return: a new set, which contains each element
11     * of elts; duplicated elements are not considered. */
12     public IntSet(int [] elts){...}
13
14     /** MODIFY this: x is added to this set if x is not present */
15     public void insert(int x){...}
16
17     /** MODIFY this: x is removed to this set if x is present
18     * @return: true if x was removed */
19     public boolean remove(int x){...}
20
21     /** @return: true if x is present in this */
22     public boolean isIn(int x){...}
23
24     /** @return: the number of elements in this */
25     public long size(){...}
26
27     /** @return: a random element in this
28     * @throws: EmptyIntSetException if this is empty */
29     public int choose() throws EmptyIntSetException {...}
30 }
```

11.1 Implementare un ADT

- usare variabili di istanza per rappresentare parti interne
- le variabili di istanza devono essere private: usare getter/setter con cura così da poter cambiare rappresentazione senza influenzare l'intero codice
- non usare variabili o metodi static: sono legati alla classe e non c'è modo di disaccoppiare il metodo dalla classe

11.2 Problemi

11.2.1 Equality

`equals()` viene ereditato da `Object`, solitamente: `obj1.equals(obj2) = true` quando gli oggetti sono immutabili e hanno lo stesso stato o quando sono mutabili e hanno la stessa identità. Si può

ridefinire `equals()` usando il polimorfismo, ridefinendo in una subclass il metodo `equals()`, ma cosa succede quando lo stato di `obj1` cambia?

11.2.2 Cloning

`obj.clone()` significa che cloniamo la struttura ad alto livello (shallow copy) o che ricorsivamente cloniamo anche le sue variabili di istanza (deep copy). `clone()` ereditato da `Object` da lo shallow copy.

11.3 Specification di un ADT

Una specification descrive cosa fa un tipo astratto (operazioni, comportamento osservabile, vincoli) senza dire come è implementato. Scopo: permettere implementazioni diverse che soddisfino la stessa semantica. L'abstract state sono i possibili valori dell'ADT. Un ADT ha operazioni, queste operazioni agiscono su un abstract state, che rappresenta ciò che l'utente percepisce della struttura, non i dettagli implementativi. Il protocol di un ADT è l'insieme delle operazioni ammesse su quel tipo astratto e delle sequenze di chiamate valide che si possono fare, cioè le regole d'uso corrette dell'oggetto secondo la sua specifica. Ogni operazione è specificata attraverso il contratto e questo si riferisce all'abstract state. La specification degli ADT dovrebbe includere la mission: cosa sa (il suo state) e cosa può fare (state transitions).

11.4 L'abstraction invariant

L'invariante di un'astrazione in Java è una proprietà logica che deve essere sempre vera per un oggetto valido, in ogni stato visibile dall'esterno, a prescindere dall'implementazione. Un invariante descrive le regole fondamentali che non devono mai essere violate, nemmeno durante l'evoluzione dello stato interno dell'oggetto.

11.5 L'abstraction function

È una mappa tra l'abstract state rappresentato dall'ADT e il concrete state, così come implementato dalle variabili di istanza, anche chiamata "rappresentazione" dell'ADT.

$$\mathcal{AF} : \text{concrete state} \rightarrow \text{abstract state}$$

Esempio con `IntSet`:

- Abstract state: insieme (nel senso matematico) di interi
- Concrete state: vettore di int
- Abstract function: $\mathcal{AF} : [1, 2] \rightarrow 1, 2$

$\mathcal{AF}$ è usata per rispondere alla domanda: "come vengono interpretate le istanze della rappresentazione dell'ADT così che rappresentino istanze dell'ADT?". Spesso $\mathcal{AF}$ è many-to-one.

11.6 Representation invariant

Il representation invariant è una condizione logica che deve essere sempre vera per ogni stato concreto di un oggetto di un ADT, in ogni momento in cui l'oggetto è "valido" dal punto di vista interno. In altre parole: è la regola che collega i dati interni effettivi alla correttezza della struttura, e che ogni operazione dell'ADT deve preservare. Il representation invariant vive nel concrete state: descrive quali configurazioni interne rappresentano abstract states validi. Esempio con `IntSet`: come rappresentiamo la cella vuota?

- shiftare a sinistra e segnare la fine del dato valido?
- creare un nuovo array?
- usare un valore specifico? Quale?

Questo invariante è associato all'abstract function. Il representation invariant dovrebbe essere descritto nella classe. Dovrebbe essere scritto prima di implementare qualsiasi metodo. Il representation invariant potrebbe essere usato dentro gli assert.

Esempio

```
1  public class Intervallo
2  {
3      private final int inizio;
4      private final int fine;
5
6      public Intervallo(int inizio, int fine)
7      {
8          if (inizio > fine)
9          {
10             throw new IllegalArgumentException("L'inizio non puo' essere
11                maggiore della fine");
12          }
13          this.inizio = inizio;
14          this.fine = fine;
15          assert repOk();
16      }
17
18      public boolean contiene(int x)
19      {
20          return x >= inizio && x <= fine;
21      }
22
23      public int getInizio()
24      {
25          return inizio;
26      }
27
28      public int getFine()
29      {
30          return fine;
31      }
32
33      // Invariante di rappresentazione: inizio <= fine
34      private boolean repOk()
35      {
36          return inizio <= fine;
37      }
38  }
```

11.7 Effetti benefici

- Quando un metodo di un ADT modifica la representation senza influenzare l'abstract state.
- È spesso fatto per motivi di performance
- Questi effetti non dovrebbero rompere l'invariante

11.8 Esporre la representation

Un ADT non deve esporre la representation. Il client potrebbe essere in grado di leggere la representation e scrivere codice che la legga. Conseguenza: il client dipende dalla representation e l'autore dell'ADT non può cambiare la representation senza richiedere che i clients cambino il loro codice.

Il client potrebbe essere in grado di modificare la representation, questo potrebbe portare alla rottura dell'invariante. Conseguenze: il client dipende dall'implementazione, l'implementatore dell'ADT dovrebbe conoscere quali cambiamenti sono possibili al concrete state.

11.9 Preservare l'invariante

- Assicurarsi che l'invariante sia vera per l'istanza memorizzata in `this` alla fine di ogni costruttore
- Per un metodo generico:
 - assumere che l'invariante valga per i parametri passati al metodo
 - assicurarsi che l'invariante valga per tutte le istanze che sono state passate al metodo, per `this` e per tutte le istanze ritornate al metodo

Questi passi devono essere effettuati per ogni metodo pubblico. "Assicurarsi" significa analizzare il codice e dedurre che in tutte le possibili esecuzioni l'invariante sia assicurato alla fine.

11.10 Operazioni

Per ogni metodo pubblico, assicurarsi che faccia quello che viene detto nel contratto. Assicurarsi che alla fine del metodo, l'abstraction function $\mathcal{AF}$ mappi la representation nel corretto abstract state. Un ADT è adeguato se le sue operazioni sono adeguate per il contesto d'uso. Difficile da determinare, dipende dal contesto. Il tipo dovrebbe essere pienamente popolato: deve essere possibile creare tutti i possibili abstract states (per esempio, per l'ADT `Poly` deve essere possibile poter creare ogni tipo di polinomio). Deve essere possibile generare una sequenza di chiamate ad operazioni così che ogni necessità di cambiamento di state possa essere ottenuto.

12 Iteration abstraction

Indica il modo in cui un linguaggio o una libreria ti permette di astrarre il meccanismo di iterazione, cioè il "come" passi attraverso una sequenza di elementi, nascondendo i dettagli concreti e rendendo il codice più dichiarativo e leggibile. In pratica, invece di dire "passo al prossimo elemento, controllo se ho finito, ripeto ...", descrivi che cosa vuoi fare su ogni elemento, lasciando alla struttura di controllo o alla funzione il compito di gestire il "come".

Esempio

Supponiamo di voler creare una classe che gestisce un set. Vogliamo che per ogni elemento sia possibile fare qualcosa.

```
1  public class IntSet
2  {
3
4      /** This class provides an ADT for sets of int.
5       *   IntSet is mutable, unbounded.
6       *   Class overview:
7       *       public IntSet()
8       *       public IntSet(int [] elts)
9       *       public IntSet(IntSet s)
10      *       public void insert(int x)
11      *       public boolean remove(int x)
12      *       public boolean isIn(int x)
13      *       private int indexOf(Integer x)
14      *       public int choose() throws EmptyIntSetException
15      *       public boolean sameValues(IntSet s2)
16      */
17  }
```

Vogliamo creare un metodo che faccia la somma tra elementi di questo set. Possibile soluzione: definire l'operazione all'interno della definizione del client.

```

1  int sum(IntSet s)
2  {
3  for ...
4  x = s.choose();
5  tmp[i] = x;
6  s.remove(x);
7  total = total + x;
8  ...
9  for (int i = 0; i < tmp.length; i++)
10 {
11 s.insert(tmp[i]);
12 }

```

Non è una buona soluzione: è inefficiente, distruttiva e non flessibile. E facendolo dentro `IntSet`? Definendo `public Vector<Integer> elements`? No. Definendo

```

1  public int[] getAllElements() // return the ints contained in elements

```

No. E se il client avesse bisogno solo alcuni degli elementi? Un altro approccio è estendendo l'ADT. Dentro `IntSet` definiamo `public int sum() {}`, qui l'implementazione può usare la rappresentazione: siamo nell'ADT, non esponiamo la sua rappresentazione ed è ok. Ma se dovessimo aver bisogno di sommare solo alcuni degli elementi? Dovremmo continuamente estendere le operazioni dell'ADT. A un certo punto dovremmo arrenderci ed esporre la sua rappresentazione. L'approccio migliore è attraverso gli iterators.

12.1 Iterators

Gli iterators sono oggetti che permettono di scorrere gli elementi di una collezione (come una List, un Set, ecc.) uno alla volta, senza esporre i dettagli interni della struttura dati. Questo è quello che il codice del client sarebbe:

```

1  ...
2  IntSet s4;
3  ...
4  // list items of s4
5  ResettableIntSetIterator it = s4.iterator();
6  for (int i = 0; it.hasNext(); i++)
7  {
8      System.out.format("\ns4[%d] = %d", i, it.next());
9  }

```

12.2 Definire iterators

Dobbiamo definire la classe

```

1  public class IntSetIterator implements Iterator<Integer>

```

Questa implementa `Iterator<Integer>`, che è un'interfaccia che specifica:

- `public boolean hasNext()`
- `public Integer next()`
- `public void remove()`

NB: il contratto di questi metodi è già definito dall'interfaccia. Struttura: Supponiamo ora di voler definire un nuovo metodo dell'ADT `IntSet`. Il nuovo metodo ritorna un iteratore inizializzato allegato al set.

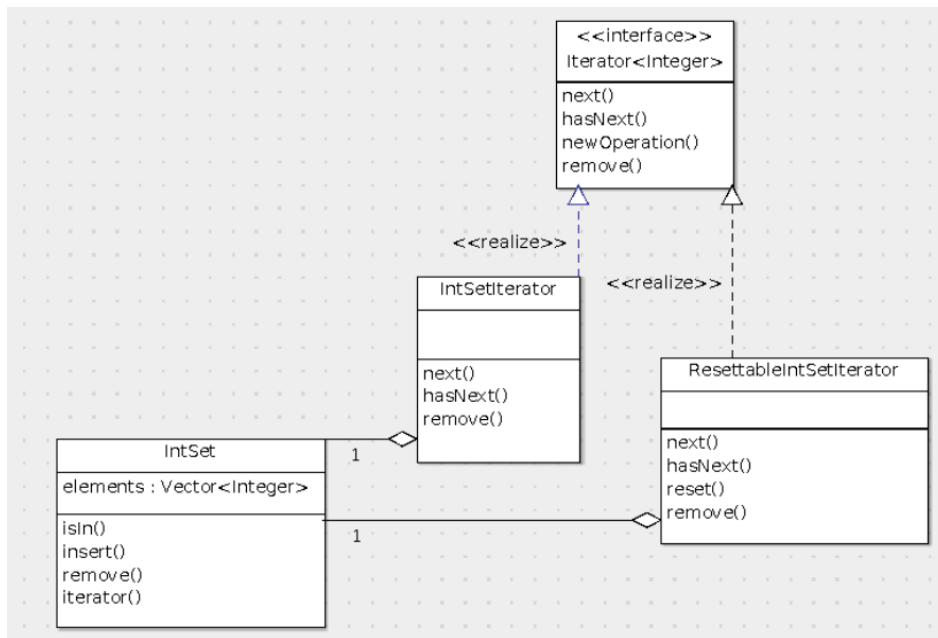


Figure 2: Struttura di una classe: IntSet

```

1  ...
2  class IntSet ...
3
4  /**
5   * @return an iterator over this set. The iterator
6   * is not sensible to mutations of this set.
7   */
8  public ResettableIntSetIterator iterator()
9  {
10     assert (this.elements!=null);
11     return (new ResettableIntSetIterator(this));
12 }
  
```

Il client userà poi l'iteratore:

```

1  // print all items
2  ResettableIntSetIterator it = s4.iterator();
3  for (int i = 0; it.hasNext(); i++)
4  {
5      System.out.format("\ns4[%d] = %d", i, it.next());
6  }
7
8  // compute the sum
9  int tot = 0;
10 it.reset();
11 for (int i = 0; it.hasNext(); i++)
12 {
13     tot = tot + it.next();
14 }
15 System.out.format("\ntot = %d", tot);
16
17 // compute the sum of x^2 foreach x: x < k
18 int tot2 = 0;
19 int k = 10;
20 it.reset();
21 for (int i = 0; it.hasNext(); i++)
  
```

```

22     {
23         Integer x = it.next();
24         if (x < k)
25         {
26             tot2 = tot2 + x * x;
27         }
28     }
29     System.out.format("\ntot2 = %d", tot2);

```

12.2.1 Implementazione

Per implementare la rappresentazione degli iterators:

```

1  /**
2   * MISSION is to provide an iterator over the elements
3   * of an IntSet.
4   * Once the iterator is created, if the original set changes
5   * the iterator continues to work on the original copy.
6   */
7  public class ResettableIntSetIterator implements Iterator<Integer>
8  {
9      /** A.F.
10       * elements contains a copy of the elements of the IntSet when
11       * this iterator was created. The data that have yet to be
12       * visited are elements[current, ...].
13       * INVARIANT:
14       * current ranges in [0,N] where N = elements.size and elements != null
15       */
16     private int current;
17     final private Vector<Integer> elements;

```

Per implementare il metodo della nuova classe:

```

1  /**
2   * Initialize the iterator
3   * with current index=0, and store a copy of the elements.
4   * @param s the source of data to initialize this iterator
5   * @throws NullPointerException if s is null
6   */
7  ResettableIntSetIterator(IntSet s)
8  {
9      if (s == null)
10     {
11         throw new NullPointerException("s cannot be null");
12     }
13     this.elements = (Vector<Integer>) s.elements.clone();
14     this.current = 0;
15 }

```

12.3 Dipendenze dell'iteratore sulla classe base

`IntSet` e `ResettableIntSetIterator` condividono alcune informazioni. Definendo l'iterator class come classe interna di `IntSet` la loro dipendenza è più localizzata. Definendo l'iterator class come privata e senza metodi extra non abbiamo bisogno di ritornare un'istanza della nuova classe, allora il client non ha bisogno di sapere se ci sono cose come "`IntSetIterator`". Conseguenze: niente dipendenza e codice più pulito. Così è come definiamo gli iterator nella classe:

```

1  private class ResettableIntSetIterator implements Iterator<Integer> {}

```

e così è come il client la userebbe:

```

1      Iterator<Integer> it2 = s4.iterator();
2      for (int i=0; it2.hasNext(); i++)
3      {
4          System.out.format("\ns4[%d] = %d", i,it2.next());
5      }
6      // compute the sum
7      it2 = s4.iterator();    // need to create a new iterator
8      tot = 0;
9      for (int i = 0; it2.hasNext(); i++)
10     {
11         tot = tot + it2.next();
12     }
13     System.out.format("\ntot = %d", tot);

```

12.4 Problemi con gli iterators

Cosa succede se modificassi gli elementi durante il loop di un iterator? In generale, quando implementiamo l'iterator dobbiamo gestire queste situazioni:

- creando una copia del dato
- lanciando un `ConcurrentModificationException` quando il count delle modifiche > 1
- facendo un `CopyOnWrite` quando la prima modifica occorre

13 Interfacce e abstract classes

In Java, i tipi possono essere definiti con:

- classi: forniscono definizione dei tipi e implementazione
- interfacce: forniscono solo definizione dei tipi
- abstract classes: forniscono la definizione del tipo e un'implementazione parziale

13.1 Interfacce

Forniscono:

- type name
- una lista di metodi (no costruttori) col corrispondente contratto

Ci sono metodi astratti o di default (astratto = niente corpo).

- non ci sono variabili di istanza, se non costanti
- non ci sono invariante e abstract function perché non c'è il concrete state

Esempio

```

1      public interface IntSet
2      {
3          public void insert(int x);
4          public boolean remove(int x);
5          public boolean isIn(int x);
6          public int size();
7          public int choose() throws EmptyIntSetException;
8          public boolean sameValues(IntSet s2);
9          public Iterator<Integer> iterator();
10         public int max();
11     }

```

Le interfacce nascondono interi tipi di famiglie, così che i client siano schermati dai dettagli. Vengono usate per ottenere dependency inversion: i clients dipendono solo dall'interfaccia e il suo contratto così che l'effetto domino durante la manutenzione sia ridotto. Una classe implementa nessuna o più interfacce. La classe:

- deve fornire tutti gli abstract methods specificati nell'interfaccia
- può fornire altri metodi
- potrebbe avere variabili di istanza

Le interfacce assicurano l'ereditarietà multipla, questo non è possibile con le classi: ereditarietà singola.

13.2 Dependency inversion

"I moduli di alto livello non devono dipendere dai moduli di basso livello. Entrambi devono dipendere da astrazioni." "Le astrazioni non devono dipendere dai dettagli. I dettagli devono dipendere dalle astrazioni." In una progettazione tradizionale, i moduli di alto livello (che contengono la logica principale del programma) dipendono direttamente da quelli di basso livello (che fanno il lavoro concreto, come scrivere su file, accedere a database, ecc.). Questo crea un forte accoppiamento. Con DIP, si inverte questa dipendenza: entrambi i moduli dipendono da un'interfaccia o astrazione. Così:

- Il modulo di alto livello non conosce i dettagli dell'implementazione.
- Il modulo di basso livello implementa l'interfaccia.

In altre parole: invece di far sì che una classe utilizzi direttamente un'altra classe concreta, fai in modo che entrambe lavorino tramite un'interfaccia. Così puoi cambiare le implementazioni senza toccare il codice "alto" che le usa. Esempio senza Dependency Inversion:

```
1  class FileLogger
2  {
3      public void log(String message)
4      {
5          System.out.println("Scrivo su file: " + message);
6      }
7  }
8
9  class OrderService
10 {
11     private FileLogger logger = new FileLogger(); // Dipendenza diretta!
12
13     public void createOrder()
14     {
15         logger.log("Ordine creato");
16     }
17 }
```

Problema: se voglio usare un logger su database o console, devo modificare `OrderService`. Esempio con Dependency Inversion:

```
1  // 1. Creo l'astrazione
2  interface Logger
3  {
4      void log(String message);
5  }
6
7  // 2. Implementazioni concrete
8  class FileLogger implements Logger
9  {
10     public void log(String message)
11     {
12         System.out.println("Scrivo su file: " + message);
13     }
14 }
```

```

13     }
14 }
15
16 class ConsoleLogger implements Logger
17 {
18     public void log(String message)
19     {
20         System.out.println("Console: " + message);
21     }
22 }
23
24 // 3. Modulo di alto livello dipende dall'astrazione
25 class OrderService
26 {
27     private Logger logger;
28
29     // Inietto la dipendenza (Dependency Injection)
30     public OrderService(Logger logger)
31     {
32         this.logger = logger;
33     }
34
35     public void createOrder()
36     {
37         logger.log("Ordine creato");
38     }
39 }

```

Ora `OrderService` non sa come il log viene scritto, sa solo che può “loggarlo” usando `Logger`. Puoi passargli qualsiasi implementazione senza cambiare il codice della classe. Come ottenerla:

- Usa interfacce o classi astratte per definire il contratto tra componenti.
- Usa Dependency Injection (costruttore, setter).
- Evita di istanziare direttamente implementazioni concrete all'interno delle classi di alto livello.
- Programma verso un'interfaccia, non verso un'implementazione.

13.3 Abstract classes

Un'abstract class fornisce un'implementazione parziale:

- alcuni metodi sono definiti come al solito
- altri sono specificati come astratti
- alcune variabili di istanza potrebbero essere specificate

Esempio

```

1 public abstract class IntSetImpl implements IntSet
2 {
3     protected int size; //
4
5     public abstract void insert(int x);
6     public abstract boolean remove(int x);
7     public abstract boolean isIn(int x);
8     public int size() {}
9     public abstract int choose() throws EmptyIntSetException;
10    public abstract boolean sameValues(IntSet s2);
11    public abstract Iterator<Integer> iterator();

```

```
12     public int max() {}  
13 }
```

13.4 Struttura tipica

- Interfacce per fornire veri generics
- Abstract classes per fornire implementazioni parziali
- Concrete classes per fornire implementazioni concrete complete

Importante: perché un'interfaccia non abbia un concrete state, il contratto dei metodi deve essere specificato solo in termini astratti e hanno bisogno di essere specificati nell'interfaccia.

13.5 Abstract classes o interfacce?

Usare le abstract classes quando:

- vuoi costruire una type family dove le classi condividono parte del comportamento
- le subclasses condividono parte dello stato

Usare le interfacce quando:

- i subtypes potrebbero non essere in relazione
- vuoi specificare comportamenti del tipo ma non vuoi fornire un'implementazione
- vuoi sfruttare l'ereditarietà multipla

14 Anonymous classes

Le anonymous classes sono classi senza nome che vengono dichiarate e istanziate in un'unica espressione. Vengono usate per creare implementazioni "al volo" di interfacce o classi astratte, specialmente quando serve una sola volta e il codice è breve. Può:

- dichiarare nuove variabili di istanza e nuovi metodi
- sovrascrivere metodi di classi base
- "vedere" variabili di istanza di classi chiuse (classi dichiarate all'interno di un'altra classe)
- "vedere" variabili locali del metodo

Vengono spesso usate per le GUI implementations. I GUI frameworks obbediscono al principio di "Hollywood": "non chiamarci, ti chiameremo noi". Tu definisci certe classi/funzioni e il framework le chiamerà in base al protocollo.

15 Object oriented design

La responsibility è un modo di descrivere il comportamento: il client richiede un certo output, l'agent è libero di eseguire le azioni che sono richieste.

- Servizio: un insieme di operazioni e informazioni che le componenti conoscono.
- Responsibility: service + contract.
- Protocollo: responsabilità di un oggetto
- Comportamento: descrizione informale del protocollo, insieme di operazioni + contratto
- Componente: l'agent che svolge il comportamento

15.1 Design principle

Una componente deve avere un piccolo insieme di responsabilità, idealmente una sola: Single Responsibility Principle (una classe dovrebbe avere solo un motivo per cambiare). Le componenti dovrebbero incapsulare i dettagli e collaborare (decomposizione del task e allocazione dei componenti in base alla delegazione).

15.2 OO Design

Centrato sull'idea che la struttura del software rispecchi quella del mondo reale.

15.3 CRC technique

Component-Responsibility-Collaboration. La CRC technique è un metodo usato nella progettazione object-oriented per identificare e organizzare le classi di un sistema prima di scrivere il codice. Aiutare a capire:

- Quali classi servono
- Quali responsabilità hanno
- Con quali altre classi collaborano

Si usano schede (card), fisiche o digitali, una per ogni classe. Su ogni scheda ci sono tre sezioni:

- Classe: nome della classe
- Responsabilità: cosa deve fare la classe, cioè i compiti e le conoscenze che possiede.
- Collaboratori: altre classi con cui interagisce per svolgere le sue responsabilità.

Quindi:

- Che nomi dare:
 - evitare nomi generici
 - evitare implementation-oriented names
 - usare il più possibile nomi concreti
- Identificare scenari reali
- Mettendo le cards su un tavolo, le componenti che collaborano dovrebbero essere messe vicine tra loro
- Simula scenari usando le cards e le responsibilities: persone diverse dovrebbero giocare il ruolo di componenti diverse
- Questo finché tutti gli scenari possono essere eseguiti e i design principles sono soddisfatti
- Se le cards diventano troppo piccole, allora le responsibilities devono essere divise tra nuove componenti con nuovi collaboratori
- Prima **cosa**, poi **chi**

16 Design patterns

Spesso usati in domini e contesti diversi:

- per creare oggetti: Factory, Abstract Factory, Builder, ...
- per creare certe relazioni: Decorator, ...
- comportamentale: Iterator, Observer, ...

16.1 Factory

Il Factory è un design pattern creazionale che delega la creazione degli oggetti a una classe/funzione dedicata, anziché usare direttamente `new` nel codice cliente. Perché usarlo: senza Factory, il codice potrebbe essere rigido:

```
1 Car car = new SportsCar(); // Dipendenza diretta da SportsCar
```

Se poi voglio cambiare in SUV o un'altra classe, dovrei modificare ovunque nel codice dove c'è `new SportsCar()`. Con una Factory:

```
1 Car car = CarFactory.createCar("sports");
```

Cambio solo dentro `CarFactory`, senza toccare il resto del codice.

Struttura

- **Product** (interfaccia o classe astratta): definisce il tipo comune degli oggetti creati.
- **ConcreteProduct**: implementazioni specifiche del Product.
- **Factory**: classe o metodo che crea istanze di **ConcreteProduct**.

Esempio

- Interfaccia comune

```
1 public interface Car
2 {
3     void drive();
4 }
```

- Implementazioni concrete

```
1 public class SportsCar implements Car
2 {
3     public void drive()
4     {
5         System.out.println("Guido una macchina sportiva!");
6     }
7 }
8
9 public class SUV implements Car
10 {
11     public void drive()
12     {
13         System.out.println("Guido un SUV robusto!");
14     }
15 }
```

- La Factory

```
1 public class CarFactory
2 {
3     public static Car createCar(String type)
4     {
5         if (type.equalsIgnoreCase("sports"))
6         {
7             return new SportsCar();
8         } else if (type.equalsIgnoreCase("suv"))
9         {
10             return new SUV();
11         }
12     }
13 }
```

```

12         throw new IllegalArgumentException("Tipo di auto
13             sconosciuto.");
14     }

```

- **Utilizzo**

```

1     public class Main
2     {
3         public static void main(String[] args)
4         {
5             Car car1 = CarFactory.createCar("sports");
6             car1.drive();
7
8             Car car2 = CarFactory.createCar("suv");
9             car2.drive();
10        }
11    }

```

Vantaggi

- **Disaccoppiamento:** il codice cliente non conosce le classi concrete.
- **Facilità di estensione:** aggiungere una nuova classe richiede solo modifiche nella factory.
- **Riutilizzo:** la logica di creazione è centralizzata.

Svantaggi

- Codice extra: una classe/metodo factory in più da mantenere.
- Complessità: per casi semplicissimi può risultare eccessivo.

16.2 Abstract Factory

L'Abstract Factory è un design pattern creazionale che fornisce un'interfaccia per creare famiglie di oggetti correlati senza specificare le loro classi concrete. Invece di avere una Factory che crea un solo tipo di prodotto, abbiamo una “super-fabbrica” che crea più tipi di prodotti correlati e garantisce che siano compatibili tra loro.

Struttura

- **AbstractFactory:** interfaccia per creare ogni tipo di prodotto della famiglia.
- **ConcreteFactory:** implementazioni che creano una variante specifica della famiglia.
- **AbstractProduct:** interfaccia comune per ciascun prodotto.
- **ConcreteProduct:** implementazioni concrete di ciascun prodotto.
- **Client:** usa solo le interfacce astratte, non conosce le classi concrete.

Esempio

Immagina un'applicazione con interfaccia grafica che può avere tema chiaro o tema scuro. Famiglia di prodotti:

- Button
- Checkbox

Abstract Factory garantisce che, se scegli “tema scuro”, entrambi i componenti appartengono a quel tema.

- Prodotti astratti

```
1      // Prodotto astratto: Button
2      public interface Button
3      {
4          void paint();
5      }
6
7      // Prodotto astratto: Checkbox
8      public interface Checkbox
9      {
10         void paint();
11     }
```

- Prodotti concreti (tema chiaro)

```
1      public class LightButton implements Button
2      {
3          public void paint()
4          {
5              System.out.println("Disegno un bottone in tema chiaro");
6          }
7      }
8
9      public class LightCheckbox implements Checkbox
10     {
11         public void paint()
12         {
13             System.out.println("Disegno una checkbox in tema chiaro");
14             ;
15         }
16     }
```

- Prodotti concreti (tema scuro)

```
1      public class DarkButton implements Button
2      {
3          public void paint()
4          {
5              System.out.println("Disegno un bottone in tema scuro");
6          }
7      }
8
9      public class DarkCheckbox implements Checkbox
10     {
11         public void paint()
12         {
13             System.out.println("Disegno una checkbox in tema scuro");
14         }
15     }
```

- Abstract Factory

```
1      public interface GUIFactory
2      {
3          Button createButton();
4          Checkbox createCheckbox();
5      }
```

- Concrete Factories

```
1 public class LightThemeFactory implements GUIFactory
2 {
3     public Button createButton()
4     {
5         return new LightButton();
6     }
7     public Checkbox createCheckbox()
8     {
9         return new LightCheckbox();
10    }
11 }
12
13 public class DarkThemeFactory implements GUIFactory
14 {
15     public Button createButton()
16     {
17         return new DarkButton();
18     }
19     public Checkbox createCheckbox()
20     {
21         return new DarkCheckbox();
22     }
23 }
```

- Client

```
1 public class Application
2 {
3     private Button button;
4     private Checkbox checkbox;
5
6     public Application(GUIFactory factory)
7     {
8         button = factory.createButton();
9         checkbox = factory.createCheckbox();
10    }
11
12    public void render()
13    {
14        button.paint();
15        checkbox.paint();
16    }
17 }
```

- Esecuzione

```
1 public class Main
2 {
3     public static void main(String[] args)
4     {
5         GUIFactory factory;
6
7         String theme = "dark"; // potrei leggerlo da config o input
8
9         if (theme.equalsIgnoreCase("light"))
10        {
11            factory = new LightThemeFactory();
12        } else {
13            factory = new DarkThemeFactory();
14        }
```

```

15
16         Application app = new Application(factory);
17         app.render();
18     }
19 }

```

16.3 Builder

Il Builder è un altro design pattern creazionale ma con lo scopo di costruire oggetti complessi passo-passo, permettendo di:

- separare la logica di costruzione dalla rappresentazione finale,
- creare diverse “varianti” dello stesso oggetto usando lo stesso processo di costruzione.

Quando usarlo

- Un oggetto ha molti parametri (alcuni opzionali),
- Vuoi evitare costruttori lunghissimi.

Senza Builder:

```

1  public class Pizza
2  {
3      private String dough;
4      private String sauce;
5      private String topping;
6
7      public Pizza(String dough, String sauce, String topping)
8      {
9          this.dough = dough;
10         this.sauce = sauce;
11         this.topping = topping;
12     }
13 }

```

Se ci fossero altri 6 parametri opzionali, il costruttore diventerebbe ingestibile, e il codice chiamante poco leggibile. Con Builder:

```

1  Pizza pizza = new Pizza.Builder()
2                .dough("integrale")
3                .sauce("pomodoro")
4                .topping("mozzarella")
5                .build();

```

Struttura

- **Product**: la classe dell’oggetto complesso.
- **Builder**: interfaccia o classe che definisce i metodi per impostare le parti.
- **ConcreteBuilder**: implementazione concreta del Builder.
- **Director** (opzionale): coordina la costruzione se il processo è fisso.
- **Client**: usa il Builder per ottenere l’oggetto.

Esempio

- Product

```
1      public class Pizza
2      {
3          private String dough;
4          private String sauce;
5          private String topping;
6
7          private Pizza(Builder builder)
8          {
9              this.dough = builder.dough;
10             this.sauce = builder.sauce;
11             this.topping = builder.topping;
12         }
13
14         public static class Builder
15         {
16             private String dough;
17             private String sauce;
18             private String topping;
19
20             public Builder dough(String dough)
21             {
22                 this.dough = dough;
23                 return this;
24             }
25
26             public Builder sauce(String sauce)
27             {
28                 this.sauce = sauce;
29                 return this;
30             }
31
32             public Builder topping(String topping)
33             {
34                 this.topping = topping;
35                 return this;
36             }
37
38             public Pizza build()
39             {
40                 return new Pizza(this);
41             }
42         }
43
44         @Override
45         public String toString()
46         {
47             return "Pizza con " + dough + ", " + sauce + " e " + topping;
48         }
49     }
```

- Uso

```
1      public class Main
2      {
3          public static void main(String[] args)
4          {
5              Pizza pizza = new Pizza.Builder()
6                          .dough("integrale")
```

```

7         .sauce("pomodoro")
8         .topping("mozzarella")
9         .build();
10
11     System.out.println(pizza);
12 }
13 }

```

Vantaggi

- Codice più leggibile e flessibile.
- Gestione naturale di parametri opzionali.
- Puoi riusare lo stesso builder per creare varianti dell'oggetto.

Svantaggi

- Più codice iniziale da scrivere.
- Non necessario per oggetti semplici.

16.4 Observer

L'Observer è un design pattern comportamentale che definisce una relazione uno-a-molti tra oggetti:

- Quando un oggetto (il Subject) cambia stato,
- Tutti gli altri oggetti interessati (gli Observers) vengono notificati automaticamente.

In breve: “tienimi aggiornato quando succede qualcosa.”

Quando usarlo

- Quando un oggetto deve notificare automaticamente più oggetti dipendenti.
- Quando vuoi disaccoppiare chi invia notifiche (Subject) da chi le riceve (Observer).

Struttura

- **Subject**: interfaccia o classe astratta che:
 - Mantiene la lista di osservatori.
 - Offre metodi per aggiungerli/rimuoverli.
 - Notifica tutti in caso di cambiamento.
- **ConcreteSubject**: implementazione del Subject.
- **Observer**: interfaccia con il metodo update() (o simile).
- **ConcreteObserver**: implementazioni specifiche che reagiscono alle notifiche.

Esempio

- Interfaccia Observer

```

1     public interface Observer
2     {
3         void update(String message);
4     }

```

- Interfaccia Subject

```
1 public interface Subject
2 {
3     void addObserver(Observer o);
4     void removeObserver(Observer o);
5     void notifyObservers(String message);
6 }
```

- ConcreteSubject

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class NewsAgency implements Subject
5 {
6     private List<Observer> observers = new ArrayList<>();
7
8     public void addObserver(Observer o)
9     {
10         observers.add(o);
11     }
12
13     public void removeObserver(Observer o)
14     {
15         observers.remove(o);
16     }
17
18     public void notifyObservers(String message)
19     {
20         for (Observer o : observers)
21         {
22             o.update(message);
23         }
24     }
25
26     // Metodo specifico di questa classe
27     public void publishNews(String news)
28     {
29         System.out.println("News: " + news);
30         notifyObservers(news);
31     }
32 }
```

- ConcreteObserver

```
1 public class NewsChannel implements Observer
2 {
3     private String channelName;
4
5     public NewsChannel(String channelName)
6     {
7         this.channelName = channelName;
8     }
9
10    public void update(String message)
11    {
12        System.out.println(channelName + " ha ricevuto la notizia
13        : " + message);
14    }
```

- Uso

```

1      public class Main
2      {
3          public static void main(String[] args)
4          {
5              NewsAgency agency = new NewsAgency();
6
7              Observer channel1 = new NewsChannel("Canale 1");
8              Observer channel2 = new NewsChannel("Canale 2");
9
10             agency.addObserver(channel1);
11             agency.addObserver(channel2);
12
13             agency.publishNews("Nuovo aggiornamento sull'economia");
14         }
15     }

```

Vantaggi

- Basso accoppiamento: il Subject non sa nulla dei dettagli degli Observer.
- Flessibilità: puoi aggiungere/rimuovere Observer in runtime.
- Implementa facilmente meccanismi reattivi/event-driven.

Svantaggi

- Possibili catene di notifiche indesiderate se ci sono molti observer collegati.
- Difficoltà nel debug quando le notifiche si propagano in modo indiretto.

16.5 Decorator

È un design pattern strutturale che serve ad aggiungere funzionalità a un oggetto in modo dinamico senza modificare il suo codice originale. È un po' come "vestire" un oggetto con altri oggetti che lo arricchiscono.

Idea di base

- Hai un'interfaccia comune (o classe astratta) che rappresenta un tipo di oggetto.
- Oltre alle implementazioni normali, crei decoratori che:
 - Implementano la stessa interfaccia.
 - Contengono un riferimento a un altro oggetto dello stesso tipo.
 - Possono aggiungere comportamento prima/dopo le chiamate delegate all'oggetto originale.

Quando usarlo

- Quando vuoi aggiungere responsabilità a un oggetto senza modificare il codice.
- Quando vuoi combinare dinamicamente comportamenti multipli.
- Alternativa alla creazione di tante sottoclassi.

Esempio

Immaginiamo un sistema di notifiche:

- Oggetto base: invia notifiche semplici via console.
- Decoratori: possono aggiungere invio via email, SMS, ecc.
- Interfaccia base

```
1 public interface Notifier
2 {
3     void send(String message);
4 }
```

- Implementazione base

```
1 public class SimpleNotifier implements Notifier
2 {
3     public void send(String message)
4     {
5         System.out.println("Notifica: " + message);
6     }
7 }
```

- Decorator astratto

```
1 public abstract class NotifierDecorator implements Notifier
2 {
3     protected Notifier wrapped;
4
5     public NotifierDecorator(Notifier wrapped)
6     {
7         this.wrapped = wrapped;
8     }
9
10    public void send(String message)
11    {
12        wrapped.send(message); // delega al componente originale
13    }
14 }
```

- Decoratori concreti

```
1 public class EmailNotifier extends NotifierDecorator
2 {
3     public EmailNotifier(Notifier wrapped)
4     {
5         super(wrapped);
6     }
7
8     public void send(String message)
9     {
10        super.send(message);
11        System.out.println("Invio email: " + message);
12    }
13 }
14
15 public class SMSNotifier extends NotifierDecorator
16 {
17     public SMSNotifier(Notifier wrapped)
18     {
```

```

19         super(wrapped);
20     }
21
22     public void send(String message)
23     {
24         super.send(message);
25         System.out.println("Invio SMS: " + message);
26     }
27 }

```

- Uso

```

1     public class Main
2     {
3         public static void main(String[] args)
4         {
5             Notifier notifier = new SimpleNotifier();
6
7             // Aggiungo funzionalita' dinamicamente
8             notifier = new EmailNotifier(notifier);
9             notifier = new SMSNotifier(notifier);
10
11             notifier.send("Promozione speciale!");
12         }
13     }

```

Vantaggi

- Flessibile: puoi combinare decoratori in qualsiasi ordine.
- Rispetta il principio di Open/Closed: puoi estendere il comportamento senza modificare il codice originale.
- Evita un'esplosione di sottoclassi.

Svantaggi

- Tanti oggetti piccoli possono rendere il debugging più complicato.
- La catena di decoratori può diventare difficile da seguire.

16.6 Composite

Il Composite è un design pattern strutturale che serve a trattare oggetti singoli e gruppi di oggetti nello stesso modo. È molto usato per rappresentare gerarchie ad albero (come file e cartelle).

Idea di base

- Hai una gerarchia di oggetti dove alcuni sono elementi semplici (foglie) e altri sono contenitori di altri oggetti.
- Vuoi che il client possa interagire con entrambi nello stesso modo, senza sapere se sta lavorando con una singola foglia o con un intero sotto-albero.

Quando usarlo

- Quando i dati sono organizzati come albero.
- Quando vuoi comportamento uniforme tra oggetti e collezioni.
- Quando vuoi evitare tanti `if` per controllare il tipo di oggetto.

Struttura

- **Component**: interfaccia comune per foglie e compositi.
- **Leaf**: implementa un elemento semplice.
- **Composite**: implementa un contenitore di Component e può avere sotto-elementi.
- **Client**: lavora con Component senza sapere se è una Leaf o un Composite.

Esempio

- Component

```
1      public interface FileSystemComponent
2      {
3          void showDetails();
4      }
```

- Leaf (File)

```
1      public class FileLeaf implements FileSystemComponent
2      {
3          private String name;
4
5          public FileLeaf(String name)
6          {
7              this.name = name;
8          }
9
10         public void showDetails()
11         {
12             System.out.println("File: " + name);
13         }
14     }
```

- Composite (Cartella)

```
1      import java.util.ArrayList;
2      import java.util.List;
3
4      public class DirectoryComposite implements FileSystemComponent
5      {
6          private String name;
7          private List<FileSystemComponent> children = new ArrayList
8              <>();
9
10         public DirectoryComposite(String name)
11         {
12             this.name = name;
13         }
14
15         public void add(FileSystemComponent component)
16         {
17             children.add(component);
18         }
19
20         public void remove(FileSystemComponent component)
21         {
22             children.remove(component);
23         }
24     }
```

```

24         public void showDetails()
25         {
26             System.out.println("Cartella: " + name);
27             for (FileSystemComponent c : children)
28             {
29                 c.showDetails();
30             }
31         }
32     }

```

- Uso

```

1         public class Main
2         {
3             public static void main(String[] args)
4             {
5                 FileSystemComponent file1 = new FileLeaf("file1.txt");
6                 FileSystemComponent file2 = new FileLeaf("file2.txt");
7
8                 DirectoryComposite folder1 = new DirectoryComposite("
9                     Documenti");
10                folder1.add(file1);
11                folder1.add(file2);
12
13                FileSystemComponent file3 = new FileLeaf("foto1.jpg");
14                DirectoryComposite folder2 = new DirectoryComposite("
15                    Immagini");
16                folder2.add(file3);
17                folder2.add(folder1); // cartella dentro cartella
18
19                folder2.showDetails();
20            }
21        }

```

Vantaggi

- Uniformità: il client non deve sapere se sta gestendo una singola foglia o un gruppo.
- Struttura ricorsiva semplice da navigare.
- Facile aggiungere nuovi tipi di componenti.

Svantaggi

- Può rendere difficile imporre restrizioni (es. una foglia non può avere figli) se tutto implementa la stessa interfaccia.
- Possibile overhead di oggetti e gestione.

16.7 Bridge

Il Bridge è un design pattern strutturale che serve a separare un'astrazione dalla sua implementazione, in modo che possano variare indipendentemente.

Idea di base

Normalmente, quando fai una classe e le sue sottoclassi, l'implementazione e l'astrazione sono legate insieme. Se però vuoi cambiare sia la gerarchia di astrazione sia la gerarchia di implementazione senza creare un'esplosione di sottoclassi, allora il Bridge ti salva. In breve: invece di avere un'unica gerarchia rigida, separi in due gerarchie collegate da un ponte (bridge).

Quando usarlo

- Quando vuoi evitare una moltiplicazione di sottoclassi per combinazioni di astrazione/implementazione.
- Quando vuoi cambiare implementazione a runtime.
- Quando vuoi isolare il codice di alto livello (astrazione) dai dettagli di basso livello (implementazione).

Struttura

- **Abstraction:** interfaccia o classe astratta di alto livello.
- **RefinedAbstraction:** estensione dell'abstraction.
- **Implementor:** interfaccia per la parte "bassa" dell'implementazione.
- **ConcreteImplementor:** implementazioni specifiche di Implementor.
- **Bridge:** l'Abstraction contiene un riferimento a un Implementor.

Esempio

- Implementor

```
1      public interface Device
2      {
3          void turnOn();
4          void turnOff();
5          void setVolume(int percent);
6      }
```

- ConcreteImplementor

```
1      public class TV implements Device
2      {
3          public void turnOn()
4          {
5              System.out.println("Accendo la TV");
6          }
7
8          public void turnOff()
9          {
10             System.out.println("Spengo la TV");
11         }
12
13         public void setVolume(int percent)
14         {
15             System.out.println("Volume TV impostato a " + percent + "
16                                 %");
17         }
18     }
19
20     public class Radio implements Device
21     {
22         public void turnOn()
23         {
24             System.out.println("Accendo la radio");
25         }
26
27         public void turnOff()
28         {
```

```

28         System.out.println("Spengo la radio");
29     }
30
31     public void setVolume(int percent)
32     {
33         System.out.println("Volume radio impostato a " + percent
34                             + "%");
35     }

```

- Abstraction

```

1     public abstract class RemoteControl
2     {
3         protected Device device;
4
5         public RemoteControl(Device device)
6         {
7             this.device = device;
8         }
9
10        public abstract void togglePower();
11        public abstract void volumeUp();
12        public abstract void volumeDown();
13    }

```

- RefinedAbstraction

```

1     public class BasicRemote extends RemoteControl
2     {
3         private boolean isOn = false;
4         private int volume = 50;
5
6         public BasicRemote(Device device)
7         {
8             super(device);
9         }
10
11        public void togglePower()
12        {
13            if (isOn)
14            {
15                device.turnOff();
16            } else
17            {
18                device.turnOn();
19            }
20            isOn = !isOn;
21        }
22
23        public void volumeUp()
24        {
25            volume = Math.min(volume + 10, 100);
26            device.setVolume(volume);
27        }
28
29        public void volumeDown()
30        {
31            volume = Math.max(volume - 10, 0);
32            device.setVolume(volume);
33        }

```

34

}

- **Uso**

```
1      public class Main
2      {
3          public static void main(String[] args)
4          {
5              Device tv = new TV();
6              RemoteControl remote = new BasicRemote(tv);
7
8              remote.togglePower();
9              remote.volumeUp();
10
11             Device radio = new Radio();
12             remote = new BasicRemote(radio);
13
14             remote.togglePower();
15             remote.volumeDown();
16         }
17     }
```

Vantaggi

- Separazione netta tra astrazione e implementazione.
- Combinazioni flessibili senza esplosione di sottoclassi.
- Possibilità di cambiare implementazione a runtime.

Svantaggi

- Aggiunge un po' di complessità in più rispetto a una soluzione semplice.
- Due gerarchie da mantenere invece di una.